



Towards Robust Deep Reinforcement Learning for Traffic Signal Control: Demand Surges, Incidents and Sensor Failures

Rodrigues, Filipe; Lima Azevedo, Carlos M.

Published in:

Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC) 2019

Link to article, DOI:

[10.1109/ITSC.2019.8917451](https://doi.org/10.1109/ITSC.2019.8917451)

Publication date:

2019

Document Version

Peer reviewed version

[Link back to DTU Orbit](#)

Citation (APA):

Rodrigues, F., & Lima Azevedo, C. M. (2019). Towards Robust Deep Reinforcement Learning for Traffic Signal Control: Demand Surges, Incidents and Sensor Failures. In *Proceedings of the IEEE Intelligent Transportation Systems Conference (ITSC) 2019* IEEE. <https://doi.org/10.1109/ITSC.2019.8917451>

General rights

Copyright and moral rights for the publications made accessible in the public portal are retained by the authors and/or other copyright owners and it is a condition of accessing publications that users recognise and abide by the legal requirements associated with these rights.

- Users may download and print one copy of any publication from the public portal for the purpose of private study or research.
- You may not further distribute the material or use it for any profit-making activity or commercial gain
- You may freely distribute the URL identifying the publication in the public portal

If you believe that this document breaches copyright please contact us providing details, and we will remove access to the work immediately and investigate your claim.

Towards Robust Deep Reinforcement Learning for Traffic Signal Control: Demand Surges, Incidents and Sensor Failures

Filipe Rodrigues¹ and Carlos Lima Azevedo¹

Abstract—Reinforcement learning (RL) constitutes a promising solution for alleviating the problem of traffic congestion. In particular, deep RL algorithms have been shown to produce adaptive traffic signal controllers that outperform conventional systems. However, in order to be reliable in highly dynamic urban areas, such controllers need to be robust with the respect to a series of exogenous sources of uncertainty.

In this paper, we develop an open-source callback-based framework for promoting the flexible evaluation of different deep RL configurations under a traffic simulation environment. With this framework, we investigate how deep RL-based adaptive traffic controllers perform under different scenarios, namely under demand surges caused by special events, capacity reductions from incidents and sensor failures. We extract several key insights for the development of robust deep RL algorithms for traffic control and propose concrete designs to mitigate the impact of the considered exogenous uncertainties.

I. INTRODUCTION

Traffic congestion is a long-standing major issue with critical consequences at the environmental, social and economic levels. For example, according to a recent study from INRIX [1], in 2018, Americans lost an yearly average of 97 hours due to congestion, costing nearly \$87 billion - an average of \$1,348 per driver. Since a significant portion of the traffic delays occurs at signalized intersections, developing efficient and adaptive traffic signal control systems is of the uttermost importance. Different control-theory based methods have been proposed and widely implemented in practice [2], [3], [4]. In recent years, several artificial intelligence techniques such as neural networks, fuzzy inference systems and reinforcement learning (RL) have also been proposed, and the exploration of the latter has seen significant progress lately [5], [6]. The fact that RL algorithms operate in a delayed return environment, where it can be difficult to understand which action leads to which outcome over many time steps, and the fact that RL agents can learn online in a dynamic environment, makes them perfectly suited for developing adaptive traffic signal control systems. More recently, the advances in deep learning and, specifically, in deep RL, have made the application of RL approaches for traffic signal control even more appealing, since they are capable of handling a large number of high-dimensional states by avoiding the discretization of the state space [7], [8].

Despite the recent rise of deep RL applications for traffic signal control, little work has been done regarding their robustness performance and uncertainty handling. de Weck

et al. [9] classified uncertainties in early system design as endogenous and exogenous. While the first can generally be influenced by the control system designer, the latter comes from the interaction of the system with the environment. Traffic control systems are also subject to both types of uncertainties. Endogenous uncertainties relate, for example, to the design of the reward function, the RL learning algorithm used, state and action space definition (see e.g. [10]), etc. Such aspects have been studied to a great extent in the traffic control literature. For example, Seyed et al. [11] compare two classes of RL algorithms: deep policy-gradient and value-function-based methods. Similarly, El-Tantawy et al. [6] provide an extensive comparison between different reward functions, state representations, actions spaces and exploration strategies. For a recent review of reinforcement learning approaches for traffic signal control, the interested reader is referred to [5].

Exogenous uncertainties, however, have been studied to a much smaller extent. These may come from the demand, the supply or other components of the traffic signaling system itself. While it is known that some variations to an *average day* traffic demand on a network are always expected, its full range of variations and its uncertain frequencies are not often explicitly considered nor handled during the system design, especially for heavy stress scenarios [12], [13]. Incidents or other road blockages may affect the supply performance of the system at stake, often leading to network effects through queue overflowing to upstream intersections [14]. Finally, from the components of the traffic signaling system, common sources of uncertainty relate to sensing and communication errors. Traffic control may rely on a few different technologies (e.g., loop detectors, cameras or RFID sensors) for its operation [15]. It is known that each of these can have different performance and failure probabilities depending on many different factors, such as weather, occlusions or computer vision classification or detection errors (cameras), communication failures, lack of maintenance, etc. [16]. All the aforementioned uncertainties can dramatically affect the performance of RL algorithms, thus making learning policies, that are robust to their effect, crucial [17], [18]. Aslani et al. [19] presented one of the few comprehensive evaluations of both discrete and continuous RL-based traffic signals in a realistic setting (Tehran’s network) for road blockages from pedestrians crossing and double parking, for high demand from non-recurring congestion and for different levels of sensor noise. Yet, a robustness analysis for deep RL techniques, where the introduction of neural networks as function approximators changes the characteristics of the learning

¹Technical University of Denmark (DTU), Bygning 116B, 2800 Kgs. Lyngby, Denmark. Emails: rodr@dtu.dk, climaz@dtu.dk

system quite dramatically [20], is still to be explored.

In this paper, we focus on the exogenous sources of uncertainty and investigate how they affect the performance of signal control systems based on deep RL. For this purpose, we develop an open-source callback-based framework for the flexible evaluation of different RL configurations under a traffic simulation environment. Using this framework, we study the effect of demand surges, incidents and sensor failures in the performance of deep RL algorithms, namely duelling deep Q-learning, for traffic signal control. Furthermore, we identify a set of initial guidelines towards the development of robust traffic signal control systems based on deep RL, whose effectiveness we demonstrate empirically thereby establishing a publicly-available benchmark for future research on this topic.

II. MODELLING FRAMEWORK

In this section we present both the modelling framework of the deep RL-based traffic signal controller and the architecture of an open-source CALLback-based REinforcement Learning framework (CAREL) to test the proposed and future deep RL-based traffic control systems (source code is available at: <http://github.com/fmpr/CAREL>).

A. Robust deep RL for traffic signal control

The general idea of RL is that of an agent interacting with an environment by observing its current state at time t , $\mathbf{s}_t \in \mathcal{S}$, taking an action $\mathbf{a}_t \in \mathcal{A}$ according to some policy π , and observing delayed rewards $r(\mathbf{s}, \mathbf{a})$, corresponding to the reward for having taken action $\mathbf{a}$ at state $\mathbf{s}$. The goal of RL is then to learn an optimal policy π^* that maximizes the discounted cumulative reward $R_t = \sum_{j=t}^{\infty} \gamma^{(j-t)} r_j$, where $\gamma \in [0, 1]$ is a discount factor that trades-off the importance of immediate and future rewards [21].

The ability of RL algorithms to correlate immediate actions with the delayed returns that they produce makes them particularly well suited for developing policies for adaptive traffic controllers that try to optimize some indicator of intersection/network performance. A particularly popular class of RL algorithms that has been proven successful for traffic signal control [7], [5], [6] is Q-Learning. Q-Learning [21] seeks to learn an optimal policy π^* implicitly by estimating the action-value function (or “Q-function”) that returns the expected future rewards by taking action $\mathbf{a}_t$ given state $\mathbf{s}_t$

$$Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \sum_{j=t}^T \mathbb{E}_\pi[r(\mathbf{s}_j, \mathbf{a}_j) | \mathbf{s}_t, \mathbf{a}_t], \quad (1)$$

and determining the best action at time t by computing $\arg \max_{\mathbf{a}_t} Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$. However, conventional Q-learning algorithms can be very limiting for traffic control, since they assume the states $\mathbf{s}_t$ to be discrete, thereby requiring a complicated process of discretization of real-valued traffic indicators (e.g. queue lengths and inductive loop data). Deep Q-learning [22] relaxes this assumption by parameterizing the potentially extremely high-dimensional Q-function with a deep neural network $Q(\mathbf{s}, \mathbf{a}; \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$ - *deep Q-network* (DQN), thus allowing the expected future rewards

for each action $\mathbf{a}$ to be approximated by a complex non-linear function of a real-valued state vector $\mathbf{s}$.

In order to improve stability and speed-up the convergence of DQN, a dueling architecture is used [23]. In dueling DQN, the Q-function $Q^\pi(\mathbf{s}, \mathbf{a}; \boldsymbol{\theta})$ is further subdivided as the sum of a value function $V^\pi(\mathbf{s}; \boldsymbol{\theta})$, capturing the overall expected reward of being in state $\mathbf{s}$ and taking probabilistic actions in future steps, and an advantage function $A^\pi(\mathbf{s}, \mathbf{a}; \boldsymbol{\theta})$, representing the advantage of choosing a particular action $\mathbf{a}$ when in state $\mathbf{s}$. The idea of dueling DQN is then to have a shared neural network architecture outputting both $V^\pi(\mathbf{s}; \boldsymbol{\theta})$ and $A^\pi(\mathbf{s}, \mathbf{a}; \boldsymbol{\theta})$ separately and estimating the Q-function as

$$Q^\pi(\mathbf{s}, \mathbf{a}) = V^\pi(\mathbf{s}; \boldsymbol{\theta}) + \left(A^\pi(\mathbf{s}, \mathbf{a}; \boldsymbol{\theta}) - \frac{1}{|\mathcal{A}|} \sum_{\mathbf{a}'} A^\pi(\mathbf{s}, \mathbf{a}'; \boldsymbol{\theta}) \right),$$

where subtracting the mean advantage has the benefit of increasing the stability of the optimization. Lastly, as a way to help mitigate the overoptimistic value estimation problem, a target network is also used [20].

So far, the RL approach described above is rather generic. In order to use it for robust traffic signal control, one must define the states, actions, rewards and neural network architecture appropriately.

State space: The state definition can play a crucial role in developing RL approaches that are robust to changes in demand, supply and sensor reliability. Furthermore, it is essential to rely on sensory data that can be easily available in a modern road network, so that the developed methodology can be readily applied in practice. Therefore, state representations based on cumulative delays [6] and vehicle-level positioning and speeds [7] were not considered. Instead, our state representation is based on queue lengths, which can be easily obtained from loop sensor- or camera-based algorithms (input-output or shock-wave models) in the sensing component of existing traffic control systems [24], [25]. Letting q_l^t denote the number of queued vehicles in lane l at time t , the state is then defined as the maximum queue length associated with each phase p :

$$s_p^t = \max_{l \in \mathcal{L}_p} q_l^t, \quad \forall p \in \{1, \dots, P\}, \quad (2)$$

where P is the number of phases and $\mathcal{L}_p$ denotes the set of lanes associated with all movements m in phase p (Fig. 2).

The advantages of the max operator in Eq. 2 are two-fold. First, it allows for a reduction of the state space in intersections with various lanes associated each phase, thereby speeding-up learning of the RL agent. Secondly, it allows for increased robustness to sensor failures, since a missing queue length measurement for a given lane will be attenuated by the max operation, rather than facing the RL agent with what would likely be a totally unseen input state. In order to further increase the robustness of the RL algorithm, we further propose extending the state representation by also including the elapsed time since the last green signal for each of the P phases. The intuition is that this extra input variables are not subjective to sensor failures and are able to provide the RL agent with auxiliary information that can

help it balance out the green assignments in case of noisy sensory inputs, while also preventing green starvation.

Action space: We consider 2 possible action spaces that are common in the literature: time-extension and phase-selection. In time-extension, the action space $\mathcal{A}$ has size $2 \times P + 1$, and it includes the possibility of keeping the current phase timings or increasing/decreasing the green duration for each phase p by 5s, as proposed in [7]. The RL agent then decides what action $a \in \mathcal{A}$ to take at the end of each cycle, which allows for a variable-length cycle but always keeps the phase sequence unaltered. In phase-selection, we have that $\mathcal{A} = \{1, \dots, P\}$ and an action $a \in \mathcal{A}$ corresponds to a transition to the selected phase with a yellow-phase transition of 3s as proposed, for example, in [8]. In order to stabilize learning and obtain a more reliable reward signal, we only allow for actions at every 10s. This has the effect of ensuring that enough evidence is collected for evaluating the reward function, while also preventing extremely short phases in a variable phase sequence setting.

Reward function: For the ease of deployment of the developed controllers, we aimed at a reward function that can also be easily obtained in real environments (unlike reward functions based on the number of stops, cumulative delays, stop time, etc.). For that purpose, the reward function used in our experiments is also based on queue lengths [6]:

$$r^t = \sum_{p=1}^P \left(\max_{l \in \mathcal{L}_p} q_l^t \right)^2 - \sum_{p=1}^P \left(\max_{l \in \mathcal{L}_p} q_l^{(t-1)} \right)^2. \quad (3)$$

We also experimented with other reward functions from the literature (see [6]) during our preliminary experiments; we consistently found the reward in Eq. 3 to be the best.

Neural network architecture: The neural network architecture used consists of 2 fully-connected hidden layers with ReLU activations, and an output layer of size P or $2 \times P + 1$, depending on whether the action space corresponds to phase-selection or time-extension, respectively. In order to prevent overfitting and increase the robustness of the controller, we further use Dropout [26] between all densely connected layers. With Dropout, units (along with their connections) are randomly dropped from the neural network during training. As it turns out, this is a key ingredient for developing controllers that are robust to, for example, sensor failures (see Section III-D). The intuition is that Dropout prevents units from co-adapting too much, thereby encouraging the network to learn multiple alternative representations that depend on different combinations of the inputs and then averaging over them in order to output a prediction (e.g. Q-values). Thus, the effect of sensor anomalies/failures can be averaged out, by leveraging information from the other sensors and the extended state representation with the elapsed times since the last green. Although it is a standard tool for preventing overfitting, this paper explicitly studies the effect of Dropout on the robustness of the learned policies to sensor failures.

Learning strategy: For the phase-selection approach, we use the Adam optimizer with a learning rate of 10^{-4} and the update rate of the target network $\tau = 0.005$. As for time-

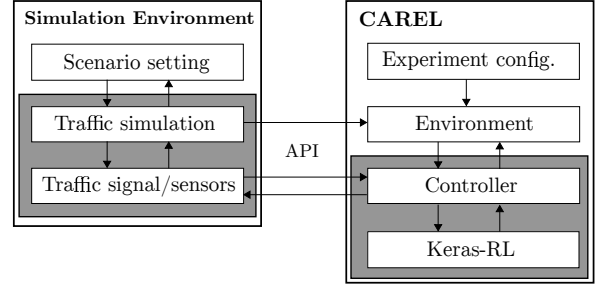


Fig. 1: CAREL framework.

extension, the best results were obtained for a learning rate of 10^{-3} and $\tau = 0.01$. For both approaches, we use experience replay with a memory size of 10000 observations. Exploration is done through an ϵ -greedy policy, with annealing for the ϵ parameter between 0.5 and 0.1 over the first 1000 steps.

B. CAREL architecture

CAREL is structured into two main objects: the Environment and the Controller (Fig. 1). The first allows for the integration with the simulation environment being used through available APIs. It enables the transfer of simulation statistics for the overall experimental assessment and for the synchronous decision making between the controller and the simulation environment. The Controller handles the learning strategy, neural network architecture, reward function, action space and state space described above. In the current Python implementation, it relies on the Keras-RL library.

With the proposed modular design, different control algorithms and configurations can be easily implemented in CAREL and transferred to other simulation or real environments. In this first deployment, the simulation environment used was AIMSUN Next [27]. Although its use in traffic signal control assessment is available in the literature (e.g.: [28]) only a couple of efforts related to deep RL-based traffic control have been reported: Casas [29] presented a deep deterministic policy gradient algorithm for the control of single intersections; and Gong et al. [30] proposed a network-level decentralized adaptive signal control based on double dueling deep Q-networks, testing it in an AM peak scenario for a suburban traffic corridor. However, no detailed provision of the integration architecture used was provided. Finally, in [31] an open-source computational framework called Flow is presented as a deep RL and control environment encapsulating traffic simulation software (including AIMSUN) for running and evaluation different automated-vehicle control scenarios. In Flow, the traffic simulation environment is called from within the deep RL environment, which has full control over the simulation environment. CAREL, however, is designed as a callback-based framework in which the simulation environment has full control over the simulation and only calls the RL-based controllers whenever an action is required or a new state has been observed. We argue that this provides a more efficient setting that is also closer to how reality works, thus making it easier to deploy the developed systems in real world environments.

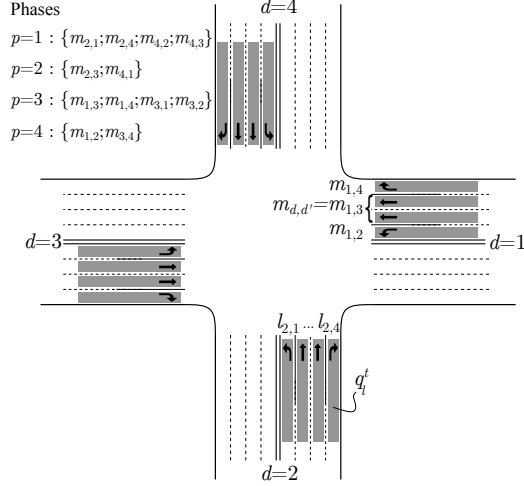


Fig. 2: Network layout.

TABLE I: Base scenario average demand

	$d = 1$	$d = 2$	$d = 3$	$d = 4$
$d = 1$	-	109.7 (47.0)	14.8 (6.1)	12.8 (6.2)
$d = 2$	167.4 (68.5)	-	88.7 (35.3)	219.2 (84.9)
$d = 3$	89.3 (36.1)	66.3 (27.7)	-	152.6 (62.5)
$d = 4$	32.2 (15.5)	64.2 (27.1)	154.2 (71.0)	-

III. EXPERIMENTS

For the evaluation of the impacts of exogenous uncertainty on the proposed deep RL control, we considered three different dimensions: demand, supply and sensing. For each of these dimensions, a set of frequent scenarios in realistic settings were considered. The full space of uncertainty was not explored, but the analysis of the deep RL behaviour and performance is here at stake. Also, in this short paper we focus our analysis on a single intersection as a first step for understanding deep RL robustness. As in previous efforts for robustness analysis [8], [19], network and coordination can have significant impacts, but is left for future research.

1) *Network*: A simple 4-leg intersection as represented in Fig. 2 was used in all experiments. Each leg or direction d has 4 lanes, $l_{d,1} \dots l_{d,4}$, each with a single movement $m_{d,d'}$, where d, d' represents the origin and destination direction. No side lanes were considered in the vicinity of the intersection. The queue length q_i^t at time t is available for each lane.

2) *Base scenario demand*: The demand for the base scenario was obtained from lane-based, 5 min loop sensor data collected by an existing adaptive traffic control system of a real intersection with the layout shown in Fig. 2. The demand by 30 min for the 7:00 to 11:00 AM period for a weekday without disruption was used as reference (Table I).

3) *Traffic Model*: AIMSUN [27] was used in current CAREL's implementation. It includes a microscopic driver behaviour model, an explicit reproduction of different traditional traffic control systems and a flexible API. AIMSUN's car-following model is based on Gipps [32], where each driver sets limits to desired braking and acceleration rates and acceleration actions are ultimately influenced by vehicle characteristics, drivers reaction time and leader's trajectory.

Vehicles can also be forced to slow down as they pass an incident to simulate the observed behaviour of drivers when passing the incident. At the intersection level, AIMSUN allows for four types of traffic signal control: unsignalised, pre-timed, actuated and external. For actuated control, a set of traditional configurations (namely minimum green, rest in red, allowance gap, passage gap) or even multi-ring or preemption configurations can be defined. For external controllers AIMSUN's API allows external changes to cycle lengths, green time durations and phase selections at each traffic light's stage. Finally, AIMSUN provides a large number of performance measures, replicating existing traffic sensing capabilities and many global performance measures usually unobservable but relevant for optimality check.

4) *Baseline Controls*: In our experiments we considered two existing baseline controllers for comparison with the deep RL-based controllers: fixed timings with optimized parameters; and actuated with a dual-ring setup and AIMSUN's default parameters [33]. For appropriate benchmark all controls relied on $p = 1, \dots, 4$ phases presented in Fig. 2.

A. Performance evaluation

For each scenario considered, three performance indicators were obtained from the simulation environment: total travel time, total delay and total stop time, by 5s and for the entire experiment duration. Each experiment accounted for a maximum of 80 episodes, each of which lasted 4h including a 10 min simulation warm-up time. This maximum number of episodes allowed for the evaluation of scenario dynamics and the stability of the deep RL performances. Finally, each experiment was replicated 10 times to allow for the quantification of the variability from both the simulation and the control algorithm intrinsic stochasticities.

1) *Computational Setting*: The processing of all controller processes within CAREL were substantially faster than AIMSUN simulations and thus suitable for real-time performance. Experiments were ran in parallel on a 16 core AMD Ryzen Threadripper 2950, 32 threads, 128 Gb of RAM under Linux.

B. Demand surge experiments

We begin by analyzing the effect of different levels of demand and demand surges in the performance of the proposed RL approach. We consider the following 3 scenarios:

Low demand: This scenario corresponds to the OD demand of the base scenario reduced by 30%.

Before event: Special events are known to cause significant disruptions in transport systems [34]. In order to simulate the potential effect of an event that is about to start in the north of the intersection, in this scenario the north-bound demand (i.e. turn movements $m_{1,4}$, $m_{2,4}$ and $m_{3,4}$) is multiplied by a factor of 2 during the 2nd and 3rd hours of the total 4-hour simulation period.

After event: This scenario is the reverse of the one above: to mimic the effect of the end of an event in a venue just north of the intersection, we multiply the incoming demand from the north direction ($d = 4$) to all other directions by a factor of 2.5 for a period of 2 hours in the middle of episode.

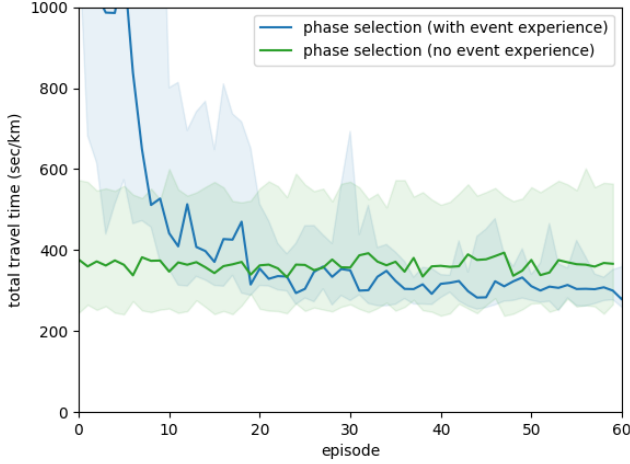


Fig. 3: Effect of event experience for “After event” scenario.

The first rows of Table II show the obtained results. Let us start by analyzing the results for the base scenario, which is based on the real demand data. As expected, we can verify that the use of an actuated controller results in better performance indicators than relying on fixed times. However, while the deep RL approach based on time-extension is barely able to outperform fixed times, we can observe that the phase-selection version significantly outperforms all the traditional approaches. This confirms the findings of previous works regarding the advantages of deep RL for traffic signal control [5], [6]. However, it questions the use of time-extension approaches as proposed for example in [7], which we could not make perform as well as the phase-selection approach regardless of the amount of hyper-parameter tuning.

What is perhaps more interesting is comparing the performance of the different methods when the demand varies. For example, we can observe that, as the demand decreases overall (“Low demand” scenario), the gap in the performance of the different methods narrows, making the phase-selection approach perform only slightly better on average than the actuated controller. This is particularly important given that a wide majority of the works researching the application of RL for traffic signal control considers fixed demands often with some arbitrary rate values [35], [8].

However, the effect of different demand levels only represents part of the problem. In order to learn RL-based controllers that are robust in lively urban environments, it is essential to account for demand surges like the ones caused by special events (e.g. music concerts, sports games, festivals or parades). In order to illustrate the importance of including scenarios demand surges during the training of the RL agent, Fig. 3 shows the result of directly deploying a RL-based controller that was learned on the base scenario on a scenario with demand surges (“After event” scenario), in comparison with the learning curve of a RL agent on the latter scenario. Not only the average behaviour of the RL agent without experience in demand surges is significantly worse, but its performance is also very unstable, with cases of extremely poor travel times as evidenced by the 10 and 90 percentiles of the curves (shaded areas).

Conveniently however, rather than re-training a new RL-agent from scratch for every possible scenario, it is possible to leverage the principles of transfer learning [36] in order to speed-up the adaptation to new scenarios. Figure 4 shows an example of this. In this experiment, we pre-trained the proposed RL-agent on the base scenario for 40 episodes and then we transferred the learned parameters to the “After event” scenario and kept training the agent on this new scenario. The left-hand side of Figure 4 shows the neural network loss, which, in the case of transfer learning, spikes at right after the change of scenario, but then quickly converges to similar loss values of those obtained by the RL agent trained from scratch on the “After event” scenario. As for Figure 4b, it shows that both approaches are able to obtain similar results over a total of 10 runs, but the pre-trained agent is able to converge within just a couple of episodes.

Lastly, by comparing the performance of the RL-based controller with fixed-times and the actuated controller (see Table II), we can verify that the phase-selection approach is the one that provides the best results for both event scenarios. However, it should be noted that, once again, the difference in performance between the various control strategies varies significantly across different scenarios.

C. Supply variation experiments

Next, we studied the effect of changes in supply, namely the effect of incidents. For this purpose, we considered the following two scenarios:

Incident A: This scenario tries to mimic the effect of a full road-block caused by an incident by setting the number of incoming vehicles from the east direction ($d = 1$) to zero for a period of 2 hours during the middle of each episode.

Incident B: In order to simulate the potential effect of an incident in a more realistic setting (i.e. using a speed reduction model, accounting for visibility, etc.), in this scenario we use the AIMSUN API to simulate an incident occupying 50m of left-most lanes from direction $d = 1$ during the 2nd and 3rd hours of the total 4-hour simulation period.

We begin by analyzing what would be the consequence of deploying on the “Incident B” scenario a RL-based controller trained on the base scenario. Figure 5 shows the obtained results in comparison with the training curve for a RL-based controller that had access to the incident scenario during learning. Similarly to the results for the demand variation, we can clearly observe a very significant deterioration of the total travel times, with extreme cases almost tripling the travel time values. This not only has a very negative impact on the level-of-service of this particular intersection, but it could potentially have a very negative impact on the entire road network.

In terms of performance of the different control strategies considered, Table II clearly shows that, when trained with experience on incident scenarios, the deep RL approach based on phase-selection is robust and it is able to significantly outperform all the other methods.

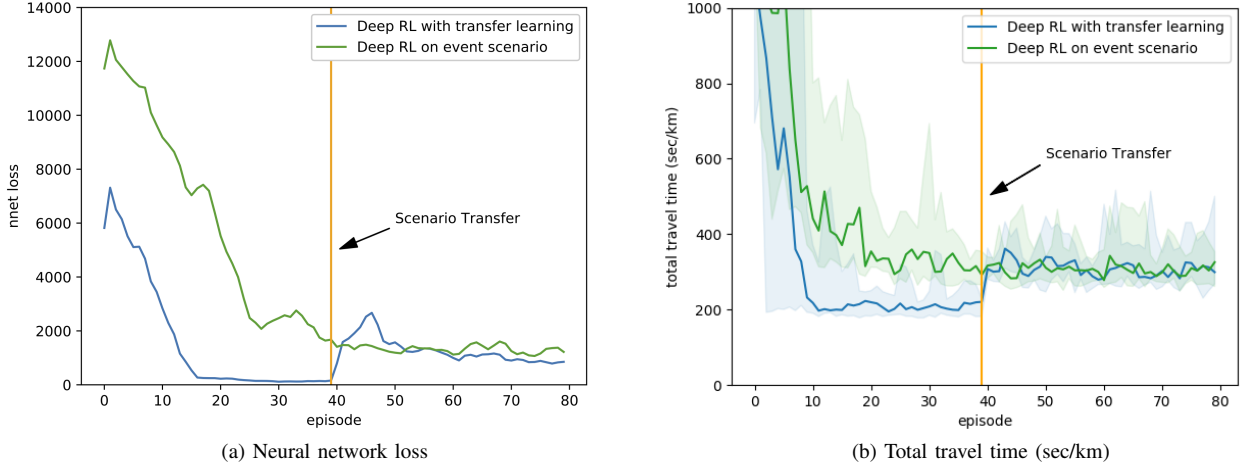


Fig. 4: Transfer learning for quick adaptation to new scenarios.

TABLE II: Results over 10 runs for scenarios with different demand and supply (average and standard deviation in seconds)

Scenario	Control Strategy	Last 30 episodes			Last 10 episodes		
		Travel Time	Delay	Stop Time	Travel Time	Delay	Stop Time
Base	Fixed Timings	341.2 ($\pm$ 0.6)	274.5 ($\pm$ 0.6)	253.7 ($\pm$ 0.6)	341.1 ($\pm$ 0.7)	274.4 ($\pm$ 0.7)	253.6 ($\pm$ 0.7)
	Actuated	299.1 ($\pm$ 5.0)	232.3 ($\pm$ 5.0)	210.0 ($\pm$ 4.72)	300.4 ($\pm$ 4.5)	233.7 ($\pm$ 4.6)	211.3 ($\pm$ 4.3)
	Time Extension	348.4 ($\pm$ 23.9)	281.7 ($\pm$ 23.9)	264.0 ($\pm$ 22.8)	331.9 ($\pm$ 6.6)	265.2 ($\pm$ 6.6)	248.4 ($\pm$ 6.4)
	Phase Selection	200.4 ($\pm$ 5.5)	133.7 ($\pm$ 5.5)	117.8 ($\pm$ 5.4)	202.0 ($\pm$ 5.3)	135.3 ($\pm$ 5.3)	119.3 ($\pm$ 5.1)
Low demand	Fixed Timings	219.6 ($\pm$ 0.9)	152.8 ($\pm$ 0.9)	138.3 ($\pm$ 0.8)	219.7 ($\pm$ 0.9)	152.9 ($\pm$ 0.9)	138.4 ($\pm$ 0.9)
	Actuated	171.1 ($\pm$ 0.7)	104.3 ($\pm$ 0.7)	90.1 ($\pm$ 0.7)	170.8 ($\pm$ 0.9)	104.1 ($\pm$ 0.9)	89.9 ($\pm$ 0.9)
	Time Extension	276.9 ($\pm$ 18.1)	210.2 ($\pm$ 18.1)	195.4 ($\pm$ 18.1)	279.6 ($\pm$ 16.7)	212.9 ($\pm$ 16.8)	198.0 ($\pm$ 16.8)
	Phase Selection	165.4 ($\pm$ 3.0)	98.6 ($\pm$ 3.0)	84.5 ($\pm$ 3.0)	168.0 ($\pm$ 2.8)	101.2 ($\pm$ 2.8)	87.1 ($\pm$ 2.7)
Before event	Fixed Timings	379.3 ($\pm$ 5.7)	312.5 ($\pm$ 5.7)	289.5 ($\pm$ 5.5)	379.9 ($\pm$ 4.9)	313.2 ($\pm$ 4.9)	290.1 ($\pm$ 4.6)
	Actuated	303.9 ($\pm$ 1.4)	237.1 ($\pm$ 1.4)	214.5 ($\pm$ 1.3)	304.2 ($\pm$ 1.2)	237.5 ($\pm$ 1.2)	214.9 ($\pm$ 1.2)
	Time Extension	370.5 ($\pm$ 16.8)	303.7 ($\pm$ 16.8)	285.3 ($\pm$ 16.3)	380.3 ($\pm$ 23.1)	313.5 ($\pm$ 23.1)	294.5 ($\pm$ 22.6)
	Phase Selection	232.6 ($\pm$ 9.8)	165.9 ($\pm$ 9.8)	148.6 ($\pm$ 9.6)	237.7 ($\pm$ 9.2)	170.9 ($\pm$ 9.2)	153.6 ($\pm$ 9.1)
After event	Fixed Timings	414.7 ($\pm$ 3.3)	347.9 ($\pm$ 3.3)	323.2 ($\pm$ 3.2)	413.6 ($\pm$ 2.9)	346.8 ($\pm$ 2.9)	322.2 ($\pm$ 2.7)
	Actuated	319.5 ($\pm$ 4.0)	252.7 ($\pm$ 4.0)	229.4 ($\pm$ 3.8)	319.4 ($\pm$ 4.7)	252.7 ($\pm$ 4.7)	229.3 ($\pm$ 4.5)
	Time Extension	466.4 ($\pm$ 34.3)	399.6 ($\pm$ 34.3)	377.3 ($\pm$ 33.2)	458.8 ($\pm$ 36.4)	392.0 ($\pm$ 36.4)	370.2 ($\pm$ 34.9)
	Phase Selection	306.3 ($\pm$ 12.5)	239.5 ($\pm$ 12.5)	218.9 ($\pm$ 11.9)	306.0 ($\pm$ 11.7)	239.3 ($\pm$ 11.7)	218.5 ($\pm$ 10.8)
Incident A	Fixed Timings	311.7 ($\pm$ 6.5)	245.0 ($\pm$ 6.5)	225.2 ($\pm$ 6.2)	311.0 ($\pm$ 5.1)	244.4 ($\pm$ 5.1)	224.5 ($\pm$ 4.9)
	Actuated	226.9 ($\pm$ 3.0)	160.3 ($\pm$ 3.0)	142.0 ($\pm$ 2.8)	227.3 ($\pm$ 3.6)	160.6 ($\pm$ 3.6)	142.4 ($\pm$ 3.4)
	Time Extension	338.5 ($\pm$ 10.5)	271.8 ($\pm$ 10.5)	254.9 ($\pm$ 10.4)	342.8 ($\pm$ 9.4)	276.2 ($\pm$ 9.4)	259.3 ($\pm$ 9.3)
	Phase Selection	194.6 ($\pm$ 7.7)	127.9 ($\pm$ 7.7)	112.8 ($\pm$ 7.7)	196.8 ($\pm$ 4.9)	130.1 ($\pm$ 4.9)	114.9 ($\pm$ 4.8)
Incident B	Fixed Timings	394.6 ($\pm$ 12.8)	327.9 ($\pm$ 12.8)	300.2 ($\pm$ 12.3)	401.8 ($\pm$ 8.6)	335.1 ($\pm$ 8.6)	307.2 ($\pm$ 8.2)
	Actuated	321.8 ($\pm$ 19.0)	255.2 ($\pm$ 18.9)	229.6 ($\pm$ 18.5)	324.0 ($\pm$ 13.3)	257.4 ($\pm$ 13.3)	231.9 ($\pm$ 13.0)
	Time Extension	363.3 ($\pm$ 9.1)	296.7 ($\pm$ 9.1)	273.6 ($\pm$ 8.9)	358.9 ($\pm$ 6.8)	292.2 ($\pm$ 6.8)	269.2 ($\pm$ 6.6)
	Phase Selection	250.1 ($\pm$ 13.2)	183.4 ($\pm$ 13.2)	160.8 ($\pm$ 13.0)	257.7 ($\pm$ 10.2)	191.1 ($\pm$ 10.2)	168.3 ($\pm$ 9.9)

TABLE III: Results over 10 runs for scenarios with sensor failures (average and standard deviation in seconds)

Scenario	Control Strategy	Failure Type	Last 10 episodes		
			Travel Time	Delay	Stop Time
Base	Fixed Timings	-	341.1 ($\pm$ 0.7)	274.4 ($\pm$ 0.7)	253.6 ($\pm$ 0.7)
	Actuated	No Failures	300.4 ($\pm$ 4.5)	233.7 ($\pm$ 4.6)	211.3 ($\pm$ 4.3)
	Time Extension	No Failures	331.9 ($\pm$ 6.6)	265.2 ($\pm$ 6.6)	248.4 ($\pm$ 6.4)
	Phase Selection	No Failures	202.0 ($\pm$ 5.3)	135.3 ($\pm$ 5.3)	119.3 ($\pm$ 5.1)
Failure A	Phase Selection (no dropout)	Before max()	314.5 ($\pm$ 13.1)	247.8 ($\pm$ 13.1)	227.3 ($\pm$ 12.6)
	Phase Selection (w/dropout)	Before max()	222.3 ($\pm$ 3.0)	155.6 ($\pm$ 3.0)	138.9 ($\pm$ 3.0)
Failure A	Phase Selection (no dropout)	After max()	355.0 ($\pm$ 18.2)	288.2 ($\pm$ 18.1)	267.4 ($\pm$ 17.6)
	Phase Selection (w/dropout)	After max()	286.3 ($\pm$ 13.7)	219.6 ($\pm$ 13.7)	201.3 ($\pm$ 13.4)
Failure B	Phase Selection (no dropout)	Before max()	258.6 ($\pm$ 18.2)	191.8 ($\pm$ 18.2)	173.9 ($\pm$ 17.7)
	Phase Selection (w/dropout)	Before max()	233.4 ($\pm$ 7.5)	166.6 ($\pm$ 7.6)	149.4 ($\pm$ 7.5)
Failure B	Phase Selection (no dropout)	After max()	436.8 ($\pm$ 52.5)	370.0 ($\pm$ 52.5)	348.7 ($\pm$ 52.0)
	Phase Selection (w/dropout)	After max()	280.8 ($\pm$ 31.0)	214.1 ($\pm$ 31.0)	195.6 ($\pm$ 30.5)
Failure C	Phase Selection (no dropout)	Before max()	222.9 ($\pm$ 3.2)	156.1 ($\pm$ 3.2)	138.6 ($\pm$ 3.1)
	Phase Selection (w/dropout)	Before max()	214.6 ($\pm$ 4.6)	147.8 ($\pm$ 4.6)	131.6 ($\pm$ 4.5)
Failure C	Phase Selection (no dropout)	After max()	241.0 ($\pm$ 4.2)	174.2 ($\pm$ 4.2)	156.8 ($\pm$ 4.0)
	Phase Selection (w/dropout)	After max()	234.3 ($\pm$ 8.8)	167.6 ($\pm$ 8.8)	149.4 ($\pm$ 8.1)

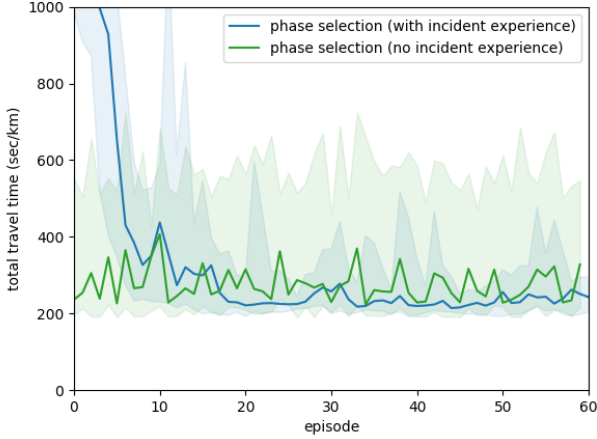


Fig. 5: Effect of incident experience for “Incid. B” scenario.

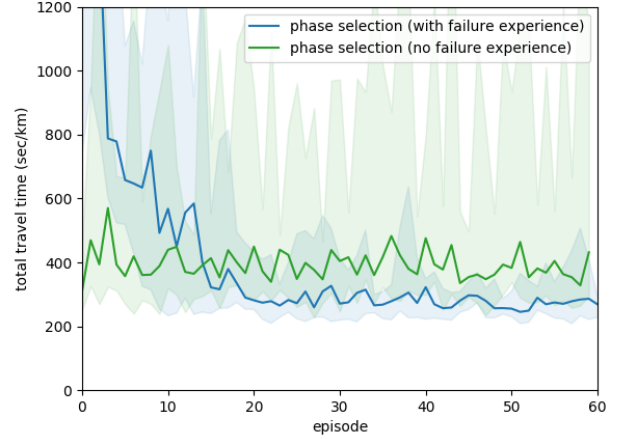


Fig. 6: Effect of failure experience for “Failure A” scenario.

D. Sensor failure experiments

Lastly, we turn our focus to robustness to sensor failures and other types of issues with sensory inputs to the deep RL agent, by considered in the following 3 scenarios:

Failure A: In order simulate realistic sensor failure patterns, in which the sensors fail in “bursts”, we implemented a 2-state Markov process for each lane sensor/group of sensors. Each sensor then alternates between two different states - OK and failure - with some probability. We further consider two failure types: before and after the max aggregation in Eq. 2. While the former could represent a problem with the individual sensors (e.g. loop detectors), the latter corresponds to the extreme setting in which all the input sensory data associated with a given phase is faulty. We assume faulty sensors to provide a value of zero (e.g. a queue length of zero). In this particular scenario, the probability of entering a failure state is set to 1%, while the probability of recovery is set to 5%. This corresponds to an overall fraction of failures of approximately 16%, and an average failure length of 20 time-steps. Kindly note that this allows for multiple sensors (eventually even all of them) to be failing simultaneously. The demand used is the same as in the base scenario.

Failure B: This scenario is similar to “Failure A”, but the probability of entering a failure state and the probability of recovery are set to 0.1% and 0.5%, respectively. Therefore, while the overall fraction of failures is the same, failures last 10 times longer.

Failure C: In this scenario, a single input sensor/group of sensors (depending on the “failure type”) is set to zero for a 2-hour period in the middle of each episode.

As with the previous experiments, we begin by analyzing the impact of input failures (scenario “Failure A”) on a RL controller trained without experiencing failures, in contrast with a RL controller that experienced failures. As Fig. 6 shows, not accounting for sensor failures during learning leads to very unreliable controllers that are very sensitive even to small changes in the inputs, which in turn can result in extremely poor performance. Interestingly, the proposed RL approach is able to learn to be robust to such failures.

In order to further understand the robustness properties of

the proposed RL approach, Table III shows the performance of the phase-selection version for the different failure scenarios described above, in comparison to the original results for the base scenario, i.e. without failures. As expected, we can verify that introducing sensor failures leads to generally worse performance of the RL approach. Also, naturally, having failures after the $\max(\dots)$ aggregation operation when computing the input state for the RL agent in Eq. 2 leads to significantly worse performance, which confirms our intuition that the max aggregation serves as an important first step towards building robust RL-based controllers for traffic signal control.

But perhaps most interestingly is observing the impact of Dropout for building robust controllers. As Table III shows, not including Dropout in the neural network leads to RL controllers that perform substantially worse when faced with scenarios with sensor failures. Please note that the same does not happen when considering only scenarios without failures, for which the versions with and without Dropout tend to perform similarly. This can be verified, for example, in the results for the “Failure C” scenario for failures “Before max()”, in which case the $\max(\dots)$ aggregation mitigates most of the effect of the failure (since it only concerns a unique lane), thus resulting only in a small difference in performance between the with/without Dropout versions.

Lastly, from a overall analysis of the results in Table III, we can observe that, when trained in scenarios with sensor failures, the RL approach can be quite robust to the widely common issue of malfunctioning sensors (e.g. inductive loop detectors or cameras). Although the average performance naturally degrades slightly, we can observe that, for all scenarios considered, the proposed deep RL approach is still able to significantly outperform fixed-time and actuated controllers that actually observe the true sensor data without any failures.

IV. CONCLUSIONS AND FUTURE WORK

This paper investigated the impact of demand surges, incidents (supply) and sensor failures on traffic signal controllers based on deep reinforcement learning, and extracted some initial, but important, insights towards the development of

deep RL approaches that are robust to many of the issues that arise in practice when being deployed in lively and highly dynamic urban environments. In order to do so, we developed a callback-based RL framework (CAREL) and integrated it with the traffic simulation software AIMSUN for testing different traffic control strategies in different scenarios that we developed and made publicly available, so that they can serve as a benchmark to foster the development of new robust RL approaches for traffic signal control in highly realistic and dynamic scenarios. Our empirical evidence shows that, in order for deep RL controllers to be robust to demand surges, incidents and sensor failures, they need to experience these scenarios during training - something that can be efficiently achieved by exploiting transfer learning. This, together with the use of Dropout and an extended state representation, allows for the development of more robust RL agents.

Despite providing an important step towards the development of robust RL-based controllers, this paper also leaves several research questions open for future work. Our ongoing work is exploring how these findings generalize to scenarios with a realistic road network with multiple intersections, and how deep RL approaches could be efficiently trained in order to be robust in more complex settings. Future work will also focus on studying how robust RL approaches based on the theory of H_∞ control [17], such as Robust Adversarial RL [18], can be used to further increase the robustness of the policies learned by the RL-based traffic signal controllers.

ACKNOWLEDGMENT

We thank Transport Simulation Systems (TSS) for supporting this research with AIMSUN licenses and feedback.

REFERENCES

- [1] INRIX Research, "Inrix global traffic scoreboard," <http://inrix.com/scorecard/>, 2019.
- [2] A. G. Sims and K. W. Dobinson, "The sydney coordinated adaptive traffic (scat) system philosophy and benefits," *IEEE Transactions on vehicular technology*, vol. 29, no. 2, pp. 130–137, 1980.
- [3] P. Hunt, D. Robertson, R. Bretherton, and R. Winton, "Scoot-a traffic responsive method of coordinating signals," Tech. Rep., 1981.
- [4] N. H. Gartner, *OPAC: A demand-responsive strategy for traffic signal control*, 1983, no. 906.
- [5] K.-L. A. Yau, J. Qadir, H. L. Khoo, M. H. Ling, and P. Komisarczuk, "A survey on reinforcement learning models and algorithms for traffic signal control," *ACM Computing Surveys*, vol. 50, no. 3, p. 34, 2017.
- [6] S. El-Tantawy, B. Abdulhai, and H. Abdelgawad, "Design of reinforcement learning parameters for seamless application of adaptive traffic signal control," *Journal of Intelligent Transportation Systems*, vol. 18, no. 3, pp. 227–245, 2014.
- [7] X. Liang, X. Du, G. Wang, and Z. Han, "Deep reinforcement learning for traffic light control in vehicular networks," *arXiv preprint arXiv:1803.11115*, 2018.
- [8] M. Schutera, N. Goby, S. Smolarek, and M. Reischl, "Distributed traffic light control at uncoupled intersections with real-world topology by deep reinforcement learning," *arXiv preprint 1811.11233*, 2018.
- [9] E. C. M. de Weck Olivier, C. P. John, et al., "A classification of uncertainty for early product and system design," *Guidelines for a Decision Supp. Method Adapted to NPD Proc.*, pp. 159–160, 2007.
- [10] M. Muresan, L. Fu, and G. Pan, "Adaptive traffic signal control with deep reinforcement learning an exploratory investigation," *arXiv preprint arXiv:1901.00960*, 2019.
- [11] S. S. Mousavi, M. Schukat, and E. Howley, "Traffic light control using deep policy-gradient and value-function-based reinforcement learning," *IET Intell. Transp. Syst.*, vol. 11, no. 7, pp. 417–423, 2017.
- [12] B. Park, C. J. Messer, and T. Urbanik, "Traffic signal optimization program for oversaturated conditions: genetic algorithm approach," *Transportation Research Record*, vol. 1683, no. 1, pp. 133–142, 1999.
- [13] Y. Tong, L. Zhao, L. Li, and Y. Zhang, "Stochastic programming model for oversaturated intersection signal timing," *Transportation Research Part C: Emerging Technologies*, vol. 58, pp. 474–486, 2015.
- [14] J.-B. Shed, Y.-H. Chou, and M.-C. Weng, "Stochastic system modeling and optimal control of incident-induced traffic congestion," *Mathematical and computer modelling*, vol. 38, no. 5-6, pp. 533–549, 2003.
- [15] X. Li and Y. Ouyang, "Reliable traffic sensor deployment under probabilistic disruptions and generalized surveillance effectiveness measures," *Operations research*, vol. 60, no. 5, pp. 1183–1198, 2012.
- [16] R. Rajagopal and P. Varaiya, "Health of californias loop detector system," *Univ. of California, Berkeley, CA, USA, Tech. Rep.*, 2007.
- [17] J. Morimoto and K. Doya, "Robust reinforcement learning," *Neural computation*, vol. 17, no. 2, pp. 335–359, 2005.
- [18] L. Pinto, J. Davidson, R. Sukthankar, and A. Gupta, "Robust adversarial reinforcement learning," in *Proc. of the 34th Int. Conf. on Machine Learning*. JMLR. org, 2017, pp. 2817–2826.
- [19] M. Aslani, M. S. Mesgari, and M. Wiering, "Adaptive traffic signal control with actor-critic methods in a real-world traffic network with different traffic disruption events," *Transportation Research Part C: Emerging Technologies*, vol. 85, pp. 732–752, 2017.
- [20] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529, 2015.
- [21] R. S. Sutton, A. G. Barto, et al., *Introduction to reinforcement learning*. MIT press Cambridge, 1998, vol. 135.
- [22] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, and M. Riedmiller, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
- [23] Z. Wang, T. Schaul, M. Hessel, H. Van Hasselt, M. Lanctot, and N. De Freitas, "Dueling network architectures for deep reinforcement learning," *arXiv preprint arXiv:1511.06581*, 2015.
- [24] H. X. Liu, X. Wu, W. Ma, and H. Hu, "Real-time queue length estimation for congested signalized intersections," *Transportation research part C: emerging technologies*, vol. 17, no. 4, pp. 412–427, 2009.
- [25] D. Ma, X. Luo, W. Li, S. Jin, W. Guo, and D. Wang, "Traffic demand estimation for lane groups at signal-controlled intersections using travel times from video-imaging detectors," *IET Intelligent Transport Systems*, vol. 11, no. 4, pp. 222–229, 2017.
- [26] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.
- [27] J. Casas, J. L. Ferrer, D. Garcia, J. Perarnau, and A. Torday, "Traffic simulation with aimsun," in *Fundamentals of traffic simulation*. Springer, 2010, pp. 173–232.
- [28] C. Vilarinho, G. Soares, J. Macedo, J. P. Tavares, and R. J. Rossetti, "Capability-enhanced aimsun with real-time signal timing control," *Procedia-Social and Behavioral Sci.*, vol. 111, pp. 262–271, 2014.
- [29] N. Casas, "Deep deterministic policy gradient for urban traffic light control," *arXiv preprint arXiv:1703.09035*, 2017.
- [30] Y. Gong, M. Abdel-Aty, Q. Cai, and M. S. Rahman, "A decentralized network level adaptive signal control algorithm by deep reinforcement learning," Tech. Rep., 2019.
- [31] C. Wu, A. Kreidieh, K. Parvate, E. Vinitzky, and A. M. Bayen, "Flow: Architecture and benchmarking for reinforcement learning in traffic control," *arXiv preprint arXiv:1710.05465*, 2017.
- [32] P. G. Gipps, "A behavioural car-following model for computer simulation," *Transportation Research Part B: Methodological*, vol. 15, no. 2, pp. 105–111, 1981.
- [33] Transport Simulation Systems (TSS), "Aimsun users manual," 2018.
- [34] F. Rodrigues, S. Borysov, B. Ribeiro, and F. Pereira, "A bayesian additive model for understanding public transport usage in special events," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 11, pp. 2113–2126, 2017.
- [35] T. Nishi, K. Otaki, K. Hayakawa, and T. Yoshimura, "Traffic signal control based on reinforcement learning with graph convolutional neural nets," *2018 21st Int. Conf. on ITS (ITSC)*, pp. 877–883, 2018.
- [36] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Transactions on knowledge and data engineering*, vol. 22, no. 10, pp. 1345–1359, 2010.